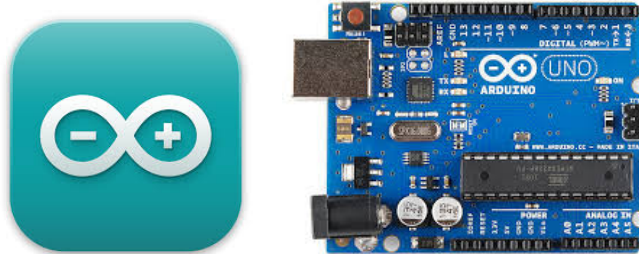


Internal Training: Arduino TexECA 15/5/2026

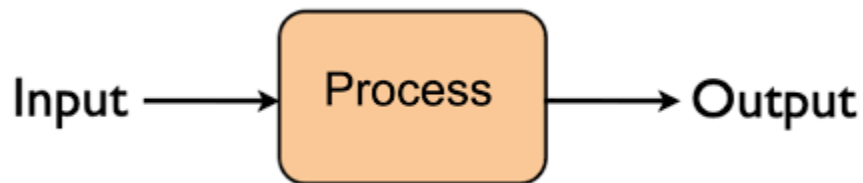
Introduction

What is Arduino?

Arduino is an open-source microcontroller platform used to build electronic and embedded systems. It allows users to read inputs from sensors/components and control outputs such as LEDs, buzzers, motors, and displays.



System Logic Flow



Input	Process	Output
Button	Logic	LED
Potentiometer	Read value	Buzzer
Ultrasonic	Distance calculation	Servo

Arduino Program Structure

A screenshot of the Arduino IDE interface. The top menu bar includes File, Edit, Sketch, Tools, and Help. Below the menu is a toolbar with icons for checking, running, and uploading code, along with a dropdown menu showing "ESP32 Wrover Module". The main editor area shows a sketch named "sketch_may9a.ino" with the following code structure:

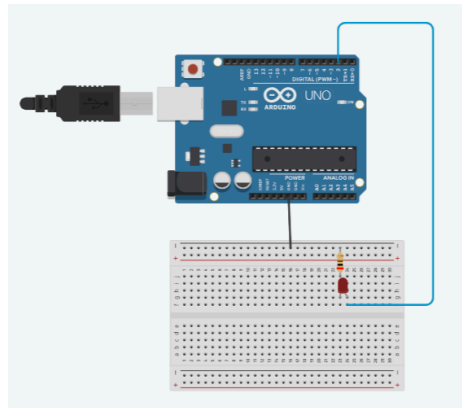
```
1 void setup() {  
2   // put your setup code here, to run once:  
3  
4 }  
5  
6 void loop() {  
7   // put your main code here, to run repeatedly:  
8  
9 }  
10
```

Modul 1: Blink & Serial Monitor

Equipment Needed:

- Arduino Uno Board
- Breadboard
- LED (Red, Yellow, Green)
- Resistor (200 Ohm) x3
- Jumper

1.1 Blink 1 LED



Circuit Diagram

IDE Code:

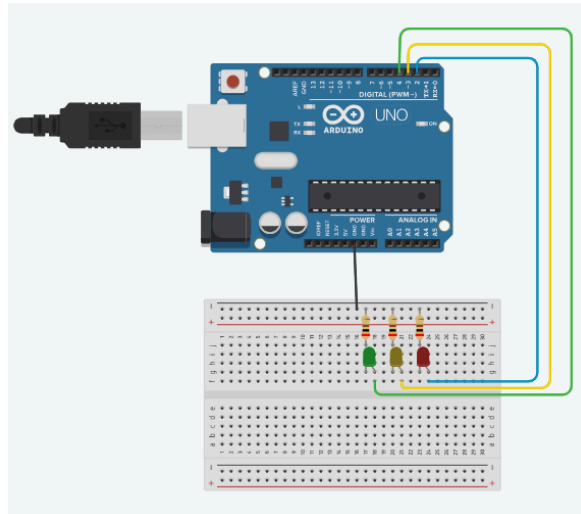
```
void setup() {  
  pinMode(2, OUTPUT);  
}  
  
void loop() {  
  digitalWrite(2, HIGH);  
  delay(500);  
  digitalWrite(2, LOW);  
  delay(500);  
}
```



Code Explanation

- pinMode() sets pin as input/output
- digitalWrite() controls HIGH/LOW
- delay() pauses execution (in millisecond)

1.2 Blink 3 LEDs



Circuit Diagram

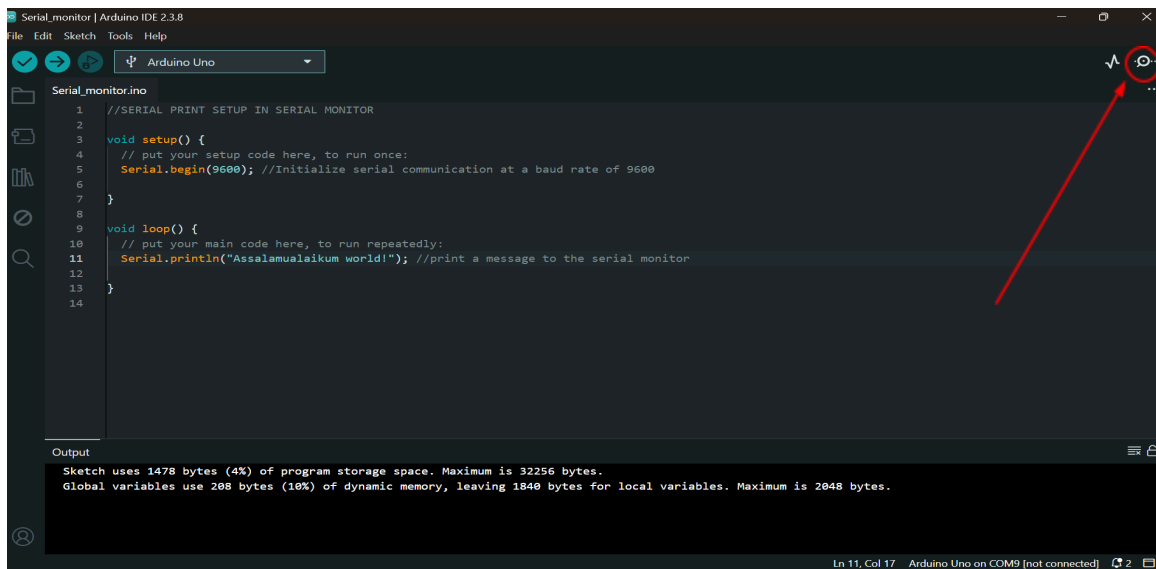
IDE Code:

```
int greenLED = 4;
int yellowLED = 3;
int redLED = 2;

void setup() {
  pinMode(greenLED, OUTPUT);
  pinMode(yellowLED, OUTPUT);
  pinMode(redLED, OUTPUT);
}

void loop() {
  digitalWrite(greenLED, HIGH);
  digitalWrite(yellowLED, LOW);
  digitalWrite(redLED, LOW);
  delay(500);
  digitalWrite(greenLED, LOW);
  digitalWrite(yellowLED, HIGH);
  digitalWrite(redLED, LOW);
  delay(500);
  digitalWrite(greenLED, LOW);
  digitalWrite(yellowLED, LOW);
  digitalWrite(redLED, HIGH);
  delay(500);
}
```

1.3 Print data to Serial Monitor



IDE Code:

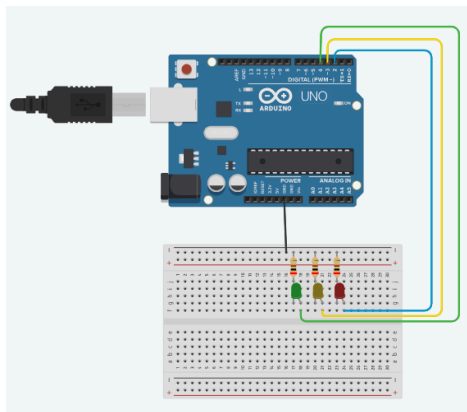
```
void setup() {  
    Serial.begin(9600);  
}  
  
void loop() {  
    Serial.println("Assalamualaikum world!");  
}
```



Code Explanation

- Serial.begin() to start serial communication between Arduino and the computer
- Serial.println() to print data to the Serial Monitor and move to a new line

1.4 Read value from Serial Monitor



Circuit Diagram

IDE Code:

```
int greenLED = 4;
int yellowLED = 3;
int redLED = 2;

void setup() {
  Serial.begin(9600);
  pinMode(greenLED, OUTPUT);
  pinMode(yellowLED, OUTPUT);
  pinMode(redLED, OUTPUT);
}

void loop() {
  digitalWrite(greenLED, HIGH);
  digitalWrite(yellowLED, LOW);
  digitalWrite(redLED, LOW);
  Serial.print(digitalRead(2));
  Serial.print(digitalRead(3));
  Serial.println(digitalRead(4));
  delay(500);
  digitalWrite(greenLED, LOW);
  digitalWrite(yellowLED, HIGH);
  digitalWrite(redLED, LOW);
  Serial.print(digitalRead(2));
  Serial.print(digitalRead(3));
  Serial.println(digitalRead(4));
  delay(500);
  digitalWrite(greenLED, LOW);
  digitalWrite(yellowLED, LOW);
  digitalWrite(redLED, HIGH);
  Serial.print(digitalRead(2));
  Serial.print(digitalRead(3));
  Serial.println(digitalRead(4));
  delay(500);
}
```



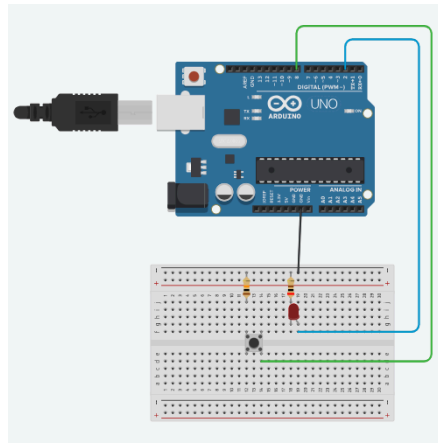
Code Explanation

- `Serial.print()` to print data to the Serial Monitor without moving to a new line
- `digitalRead()` to read digital input values from sensors

Modul 2: Button

Equipment Needed:

- Arduino Uno Board
- Breadboard
- Button
- LED
- Resistor (10k Ohm)
- Resistor (200 Ohm)
- Jumper



Circuit Diagram

IDE Code:

```
int led = 2;
int button = 8;

void setup() {
  pinMode(button, INPUT_PULLUP);
  pinMode(led, OUTPUT);
}

void loop() {
  int buttonstate=digitalRead(button);
  digitalWrite(led, !buttonstate);
}
```



Code Explanation

- INPUT_PULLUP used to enable Arduino's internal pull-up resistor

Normally:

- button not pressed → HIGH
- button pressed → LOW

Task 1

Make the button latch. Press the button once and the LED will hold the state. Press again to turn off.

Task 2

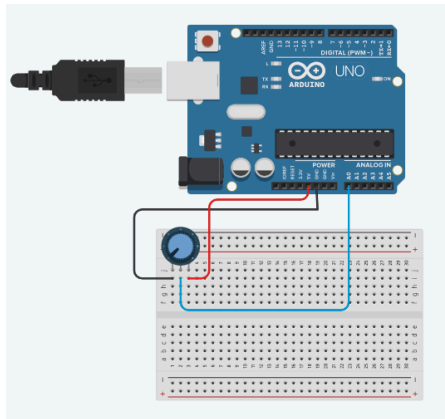
Modify the code from Task 1 so that we can observe the button state value with Serial Monitor.

Modul 3: Potentiometer & Buzzer

Equipment Needed:

- Arduino Uno Board
- Breadboard
- Potentiometer
- Buzzer
- Jumper

3.1 Potentiometer



Circuit Diagram

IDE Code:

```
void setup()
{
  Serial.begin(9600);
  pinMode(A0, INPUT);
}

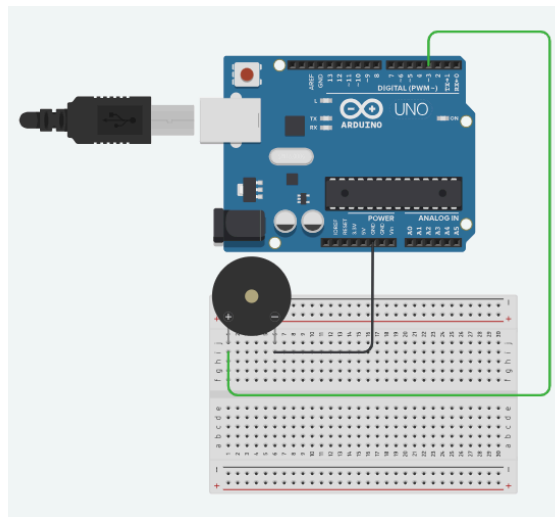
void loop()
{
  int potValue = analogRead(A0);
  Serial.println(potValue);
}
```



Code Explanation

- `analogRead()` to read analog input values from sensors/components

3.2 Buzzer



Circuit Diagram

IDE Code:

```
int buzzer = 3;

void setup() {
  pinMode(buzzer, OUTPUT);
}

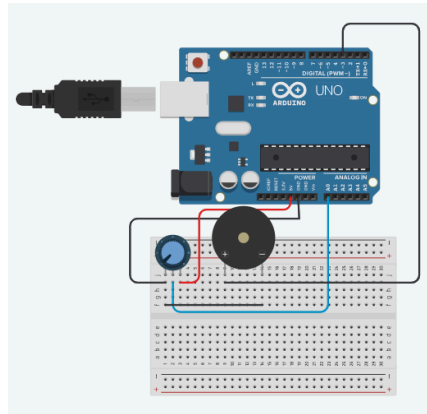
void loop() {
  analogWrite(buzzer,0); //write analog value to buzzer (0-255)
  delay(50);
  analogWrite(buzzer,51);
  delay(50);
  analogWrite(buzzer,102);
  delay(50);
  analogWrite(buzzer,204);
  delay(50);
  analogWrite(buzzer,255);
  delay(50);
}
```



Code Explanation

- `analogWrite()` used to output PWM (Pulse Width Modulation) signal

3.3 Potentiometer + Buzzer



Circuit Diagram

IDE Code:

```
void setup()
{
  pinMode(3, OUTPUT);
  pinMode(A0, INPUT);
}

void loop()
{
  int potValue = analogRead(A0);
  potValue = map(potValue, 0, 1023, 0, 255);
  analogWrite(3, potValue);
}
```



Code Explanation

- `map()` used to convert a value from one range to another range

Mini Project: Traffic Light

Requirements:

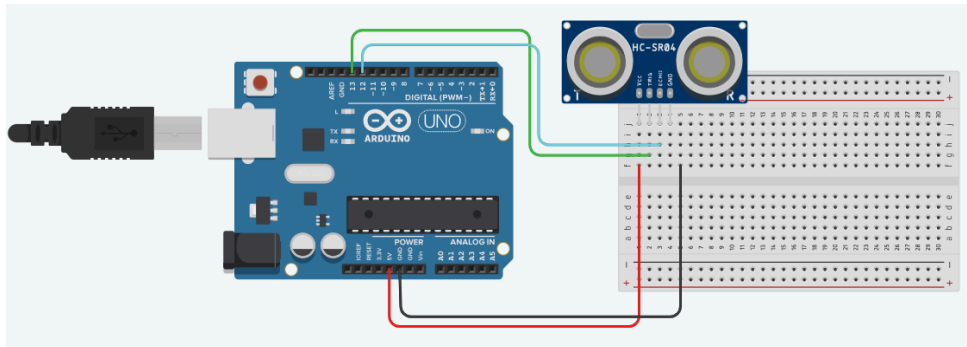
- Initially, the Green LED (for vehicle) and Red LED (for pedestrian) is turn on
- When the button is pressed, The Green LED (for vehicle) will blink for about 3 seconds before turning to Yellow for 2 seconds and Red
- When the LED (for vehicle) is turning Red, the Green LED (for pedestrian) is on for about 5 seconds before turning Red
- When the LED (for pedestrian) is turning Red, the Green LED (for vehicle) is turn on (just like in initial state)

Modul 4: Ultrasonic & Servo

Equipment Needed:

- Arduino Uno Board
- Breadboard
- Ultrasonic sensor
- Servo
- Jumper

4.1 Ultrasonic



Circuit Diagram

IDE Code:

```
const int trigPin = 13;
const int echoPin = 12;

void setup()
{
  Serial.begin(9600);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
}

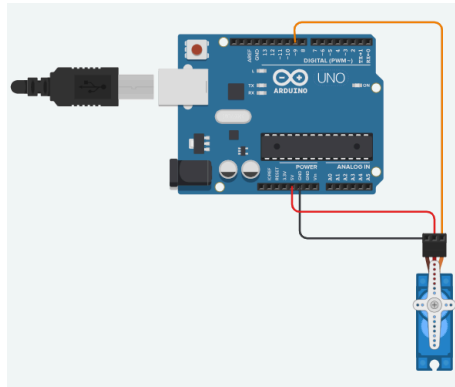
void loop()
{
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  long duration = pulseIn(echoPin, HIGH);
  int distance = duration*0.034/2;
  Serial.print("Distance (cm): ");
  Serial.println(distance);
  delay(1000);
}
```



Code Explanation

- `delayMicroseconds()` pauses the program for a number of microseconds
- `pulseIn()` used to measure how long a pulse stays HIGH or LOW

4.2 Servo



Circuit Diagram

IDE Code:

```
#include <Servo.h>

Servo myservo; //create servo object to control servo

void setup()
{
  myservo.attach(9);
}

void loop()
{
  static int pos;
  for(pos=0;pos<=180;pos++)
  {
    myservo.write(pos); //tells servo to go to the position
    given
    delay(15);
  }
  for(pos=180;pos>=0;pos--)
  {
    myservo.write(pos); //tells servo to go to the position
    given
    delay(15);
  }
}
```



Code Explanation

- `myservo.attach()` connects the servo object to a pin
- `myservo.write()` used to rotate servo to a specific angle

Mini Project: Autogate

Requirements:

- The servo will rotate 90 degrees when object is detected within 5 cm
- When object out of range, the servo will rotate back to its initial position after 3 seconds